

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	7-MPI并行应用	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.5.14

实验目的

- 掌握MPI并行的原理
- 掌握将串行程序修改为MPI并行程序的方法
- 通过将程序修改为并行程序来提升性能

实验过程与核心代码

修改Main函数

```
// initialize mpi
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

首先初始化MPI环境，获得rank和size.

```
if ( first ){
    if(rank == 0){
        // initialize
        for ( i = 0; i < 2 * n; i = i + 2 ){
            z0 = ggl ( &seed );
            z1 = ggl ( &seed );
            x[i] = z0;
            z[i] = z0;
            x[i+1] = z1;
            z[i+1] = z1;
        }
    }
}
```

随后在进程0中初始化数据。

```

// Transform forward, back
if ( first ){
    MPI_Barrier(MPI_COMM_WORLD);

    sgn = + 1.0;
    cfft2 ( n, x, y, w, sgn );
    MPI_Barrier(MPI_COMM_WORLD);
    sgn = - 1.0;
    cfft2 ( n, y, x, w, sgn );
    MPI_Barrier(MPI_COMM_WORLD);
    // Calculate error
    fnm1 = 1.0 / ( double ) n;
    error = 0.0;
    for ( i = 0; i < 2 * n; i = i + 2 ){
        error = error
            + pow ( z[i] - fnm1 * x[i], 2 )
            + pow ( z[i+1] - fnm1 * x[i+1], 2 );
    }
    error = sqrt ( fnm1 * error );
    cout << " " << setw(12) << n
        << " " << setw(8) << nits
        << " " << setw(12) << error;
    first = 0;
}

else{
    ctime1 = cpu_time ( );
    for ( it = 0; it < nits; it++ ){
        MPI_Barrier(MPI_COMM_WORLD);
        sgn = + 1.0;
        cfft2 ( n, x, y, w, sgn );
        MPI_Barrier(MPI_COMM_WORLD);

        sgn = - 1.0;
        cfft2 ( n, y, x, w, sgn );
        MPI_Barrier(MPI_COMM_WORLD);
    }
}

```

在调用并行化的cfft2函数前，使用MPI_Barrier同步进程。

cfft2函数

```

double **y_prc = (double **)malloc( 2*n_prc * sizeof(double));

MPI_Scatter(x, 2*n_prc, MPI_DOUBLE, x_prc, 2*n_prc, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
MPI_Scatter(y, 2*n_prc, MPI_DOUBLE, y_prc, 2*n_prc, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

// 2D FFT
step(n_prc, mj, &x_prc[0*2+0], &x_prc[(n_prc/2)*2+0], &y_prc[0*2+0],
&y_prc[mj*2+0], w, sgn);

```

首先用MPI_Scatter向各个进程发放数据。

在调用step时，使用分块后的数据。

```

        mj = n / 2;
        step(n_prc, mj, &x_prc[0*2+0], &x_prc[(n_prc/2)*2+0], &y_prc[0*2+0],
        &y_prc[mj*2+0], w, sgn);

        // ÉÕƎˆËùÓðœ³ìµÄæÆËäá¹ûµæžùœ³ì
        MPI_Gather(x_prc, 2*n_prc, MPI_DOUBLE, x, 2*n_prc, MPI_DOUBLE, 0,
        MPI_COMM_WORLD);
        MPI_Gather(y_prc, 2*n_prc, MPI_DOUBLE, y, 2*n_prc, MPI_DOUBLE, 0,
        MPI_COMM_WORLD);

```

调用结束后，使用MPI_Gather收集数据。

使用Valgrind massif工具

安装valgrind massif后，使用命令：

```
valgrind --tool=massif --stacks=yes ./lab7 运行程序。
```

实验结果

并行化后：

```

524288      1      0      1048576      1      0patmon@PCLa
b1:~$ mpirun -np 2 ./lab7
14 May 2025 04:53:24 PM
14 May 2025 04:53:24 PM

FFT_SERIAL
C++ version

Demonstrate an implementation of the Fast Fourier Transform
of a complex data vector.

Accuracy check:

    FFT ( FFT ( X(1:N) ) ) == N * X(1:N)

      N      NITS      Error      Time      Time/Call      MFLOPS
6      2      10000      0.346937      0.243565      1.21783e-05      0.82113
1      4      10000      0.728528      0.238634      1.19317e-05      3.3524
1      8      10000      0.658079      0.247188      1.23594e-05      9.7092
1     16      10000      0.738681      0.262177      1.31088e-05      24.41
1     32      1000      0.820359      0.024218      1.2109e-05      66.066

```

	N	NITS	Error	Time	Time/Call	MFLOPS
3	128	1000	0.766989	0.025285	1.26425e-05	354.3
6	256	1000	0.829825	0.025932	1.2966e-05	789.75
8	512	100	0.812277	0.002442	1.221e-05	1886.9
8	1024	100	0.823581	0.002648	1.324e-05	3867.0
7	2048	100	0.824638	0.002503	1.2515e-05	9000.
4	4096	100	0.818202	0.002982	1.491e-05	16482.
9	8192	10	0.814795	0.000335	1.675e-05	31789.
9	16384	10	0.815816	0.000334	1.67e-05	68675.
4	32768	10	0.817066	0.00075	3.75e-05	6553
6	65536	10	0.815902	0.001283	6.415e-05	81728.
4	131072	1	0.816448	0.001326	0.000663	16804.
1	262144	1	0.816608	0.004027	0.0020135	11717.
4	524288	1	0.816189	0.006108	0.003054	16308.
9	1048576	1	0.816323	0.017208	0.008604	12187.

```
FFT_SERIAL:
Normal end of execution.
```

并行化前:

```
palmon@PCLab1:~$ ./ftt
14 May 2025 04:19:15 PM
```

FFT_SERIAL						
C++ version						
Demonstrate an implementation of the Fast Fourier Transform of a complex data vector.						
Accuracy check:						
$\text{FFT}(\text{FFT}(X(1:N))) == N * X(1:N)$						
	N	NITS	Error	Time	Time/Call	MFLOPS
3	2	10000	7.85908e-17	0.000671	3.355e-08	298.06
3	4	10000	1.20984e-16	0.001807	9.035e-08	442.72
5	8	10000	6.8208e-17	0.002734	1.367e-07	877.83
7	16	10000	1.43867e-16	0.009574	4.787e-07	668.47
4	32	1000	1.33121e-16	0.001416	7.08e-07	1129.9
5	64	1000	1.77654e-16	0.006532	3.266e-06	587.87
4	128	1000	1.92904e-16	0.012384	6.192e-06	723.51

1						
	512	100	1.92749e-16	0.003251	1.6255e-05	1417.4
1						
	1024	100	2.31209e-16	0.007283	3.6415e-05	1406.0
1						
	2048	100	2.44501e-16	0.015134	7.567e-05	1488.5
7						
	4096	100	2.47659e-16	0.034039	0.000170195	1443.9
9						
	8192	10	2.57483e-16	0.007477	0.00037385	1424.3
1						
	16384	10	2.7342e-16	0.015792	0.0007896	1452.4
8						
	32768	10	2.9187e-16	0.032732	0.0016366	1501.6
5						
	65536	10	2.83474e-16	0.076195	0.00380975	1376.1
7						
	131072	1	3.14377e-16	0.015378	0.007689	1448.9
7						
	262144	1	3.21308e-16	0.03261	0.016305	1446.9
8						
	524288	1	3.2835e-16	0.067513	0.0337565	1475.4
9						
	1048576	1	3.28552e-16	0.174021	0.0870105	1205.1
1						
FFT_SERIAL:						
Normal end of execution.						

经过并行化后，运行时间相比并行化之前减少了。

线程数	并行化前	2	4	8	16
N = 1048576, NITS=1	0.17	0.017	0.081	0.013	0.08



Valgrind massif运行结果如上图。从图中可以看出，随着程序运行时间增加，内存消耗量也增加，最多到达56.02MB。

实验感想

在本次实验中，我学会了如何将一个串行程序改为MPI并行程序。这需要使用MPI_Scatter发放数据，使用MPI_Gather收集数据。同时在实验中，我遇到了很多问题。首先，我已经忘记了什么是快速傅里叶变换，看不懂源代码。其次，我在写代码时又遇到了段错误的问题。通过查阅博客与逐块排查，我解决了这两个问题。